PAPER TRADING ENGINE — SLICE 2 · LECTURE NOTES

Ownership & the Rule of Five

Why `OrderBook` refuses to be copied, and why it is still allowed to move — read through the one detail that makes the difference: an `OrderLocator` is not a copy of an order, it is a pointer to one.

Fuaad Bashi Shurie August 2026 OrderBook · include/te/book/order_book.hpp

---

BY THE END OF THIS NOTE

- You can explain, from the actual memory layout, why a compiler-generated copy of `OrderBook` would silently produce two books that disagree with each other.
- You know precisely what `= delete` and `= default` each promise the compiler, and why both directions of the policy have to be spelled out together.
- You can name the same ownership pattern the next time it appears in this codebase — it already has, twice.

1 The relationship inside `OrderBook`

This lecture exists because of one design fact: `OrderBook` contains pointers in disguise. Its `OrderLocator` objects hold iterators pointing into the book's own lists.

Suppose the book contains order 42. The price ladder holds the real thing:

```
bids_[Price{100}]
    -> PriceLevel
        -> std::list<RestingOrder>
            -> RestingOrder{ OrderId{42}, Qty{5} }
```

The id index holds a second way to reach the exact same order:

```
orderIndex_[OrderId{42}]
    -> OrderLocator
        side      = buy
        price     = 100
        order_pos = iterator into the list above
```

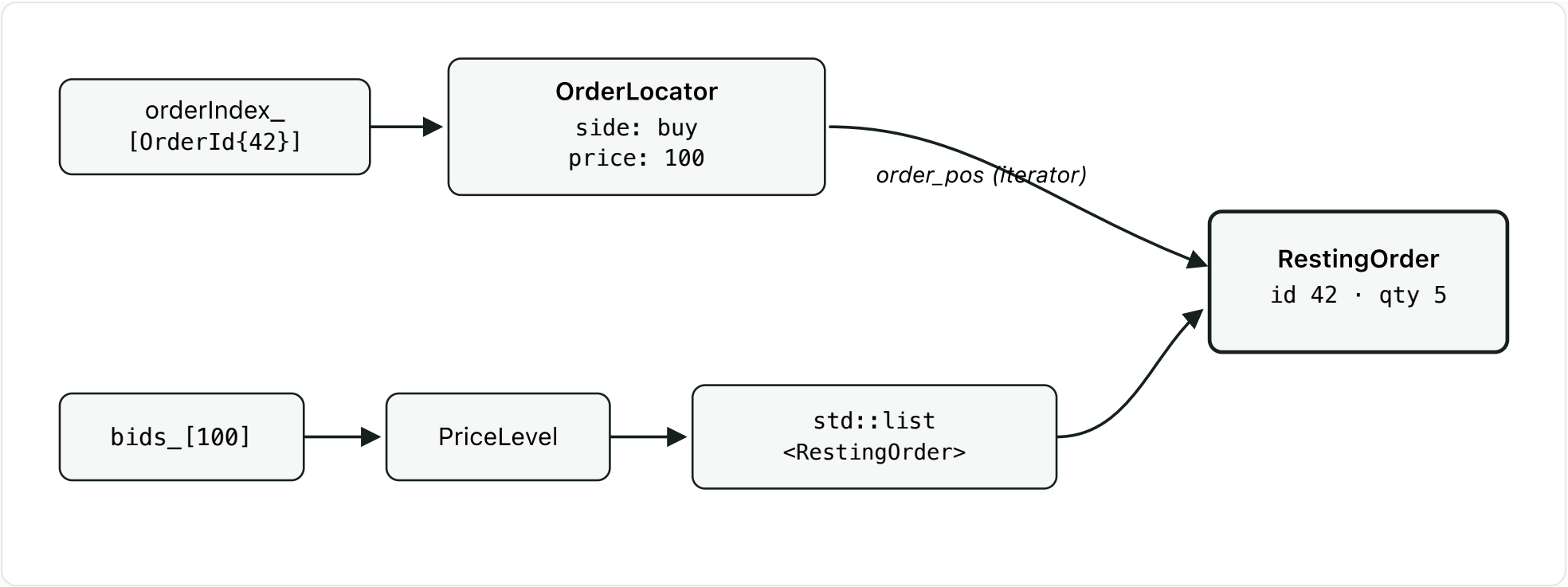

Figure 1. Two independent paths, one node. The ladder reaches order 42 through `bids_ → PriceLevel → list`; the index reaches the same order through `orderIndex_ → OrderLocator → order_pos`. `order_pos` is not a copy — it is an iterator into the list the ladder already owns.

This is what makes Modify and Remove *O(1)*: given only an `OrderId`, the book jumps straight to the node instead of scanning every price level for it.

2 What the compiler would do by default

If we had written no copy rules at all, C++ would generate them for us. That would silently permit:

```
te::OrderBook original;
// ... orders added to original ...

te::OrderBook copy{original};   // compiler-generated copy constructor
```

The generated constructor copies each member in turn — `bids_`, `asks_`, `orderIndex_`. The maps and lists do the honest thing: they allocate brand-new nodes for `copy`, holding equal values. But an iterator is a different kind of member. Copying an iterator only copies *where it points* — it has no way to know a new list exists, let alone retarget itself toward the matching node inside it.

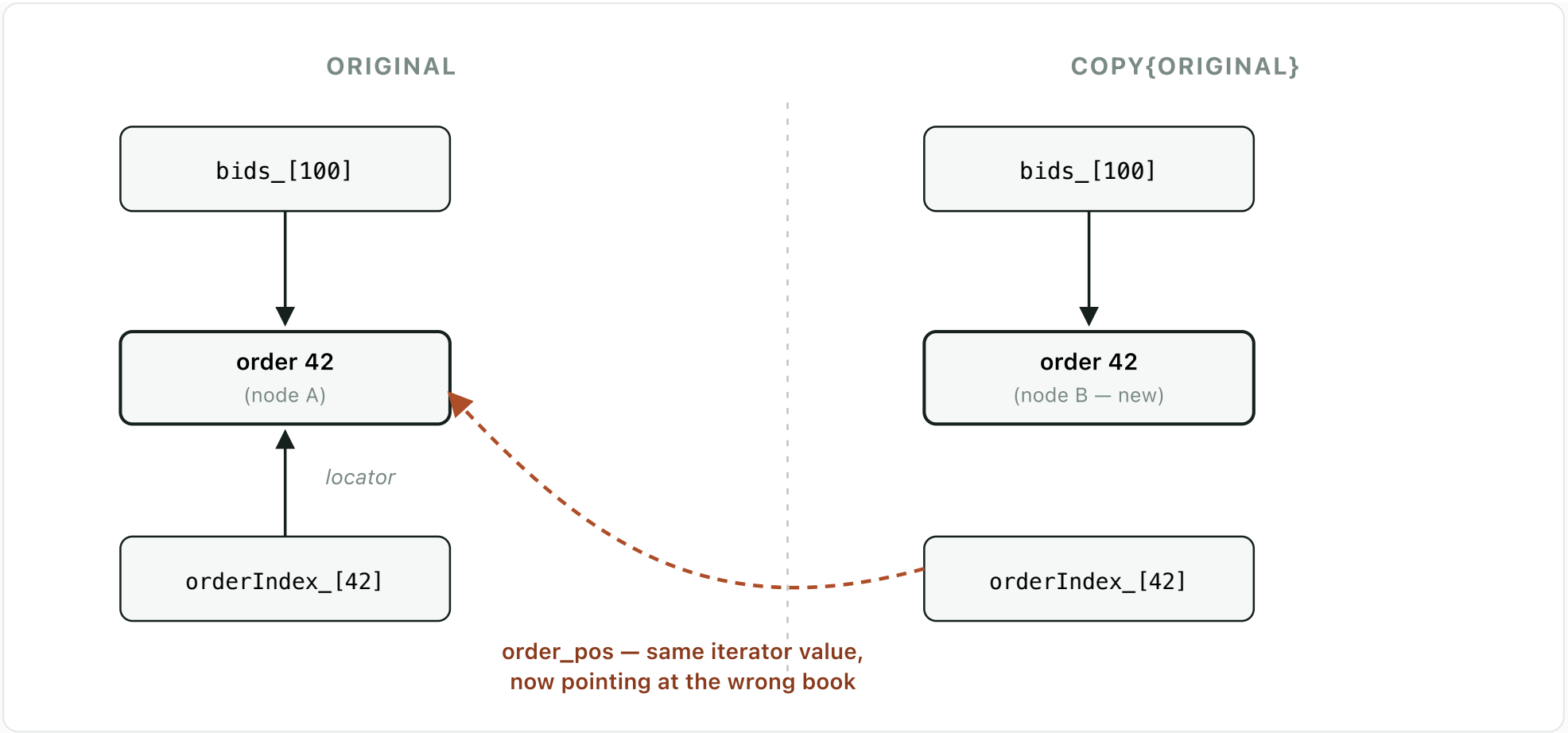

Figure 2. The map halves are correctly, independently duplicated — `copy.bids_` genuinely owns node B. But `copy.orderIndex_[42].order_pos` was only ever a copied iterator *value*; it still resolves to node A, inside `original`.

CONSEQUENCE

The copied book's two views of its own state now disagree. `copy.qtyAt(...)` reads `copy.bids_` and looks correct. But Modify or Remove on order 42 goes through `copy.orderIndex_`, follows `order_pos`, and touches `original`**'s** order instead. If `original` has since been destroyed, that iterator is dangling, and dereferencing it is undefined behaviour: it might crash, corrupt memory, silently edit the wrong book — or simply appear to work during testing and fail later. That last possibility is the dangerous one.

3 What `= delete` means

The fix is two lines in `order_book.hpp:45–46`:

```
OrderBook(const OrderBook&)            = delete;   // copy construction
OrderBook& operator=(const OrderBook&) = delete;   // copy assignment
```

These forbid two distinct operations. **Copy construction** is building a new object from an existing one:

```
te::OrderBook first;
te::OrderBook second{first};   // compiler error
```

Copy assignment is overwriting an already-existing object with another's state:

```
te::OrderBook first;
te::OrderBook second;
second = first;                // compiler error
```

DEFINITION

`= delete` tells the compiler: *this operation is deliberately forbidden — if any code attempts it, stop compiling.* Figure 2's failure mode was a *runtime* bug that might not show up until production. Deleting the operation turns it into a *compile-time* one, which is strictly better: it fails on every build, for every caller, before the program ever runs.

4 Why moving is still allowed

Copying and moving answer different questions. Copying asks for two independent books with equivalent state. Moving asks for one book's state to be handed to a new owner:

```
te::OrderBook original;
// ... orders added ...

te::OrderBook destination{std::move(original)};
```

With the standard allocators used here, moving a `std::map`, `std::list`, or `std::unordered_map` transfers their existing nodes rather than duplicating every one of them. No node changes address. So every `order_pos` iterator is *still correct* after the move — the node it points to now belongs to `destination`, but it is the very same node.

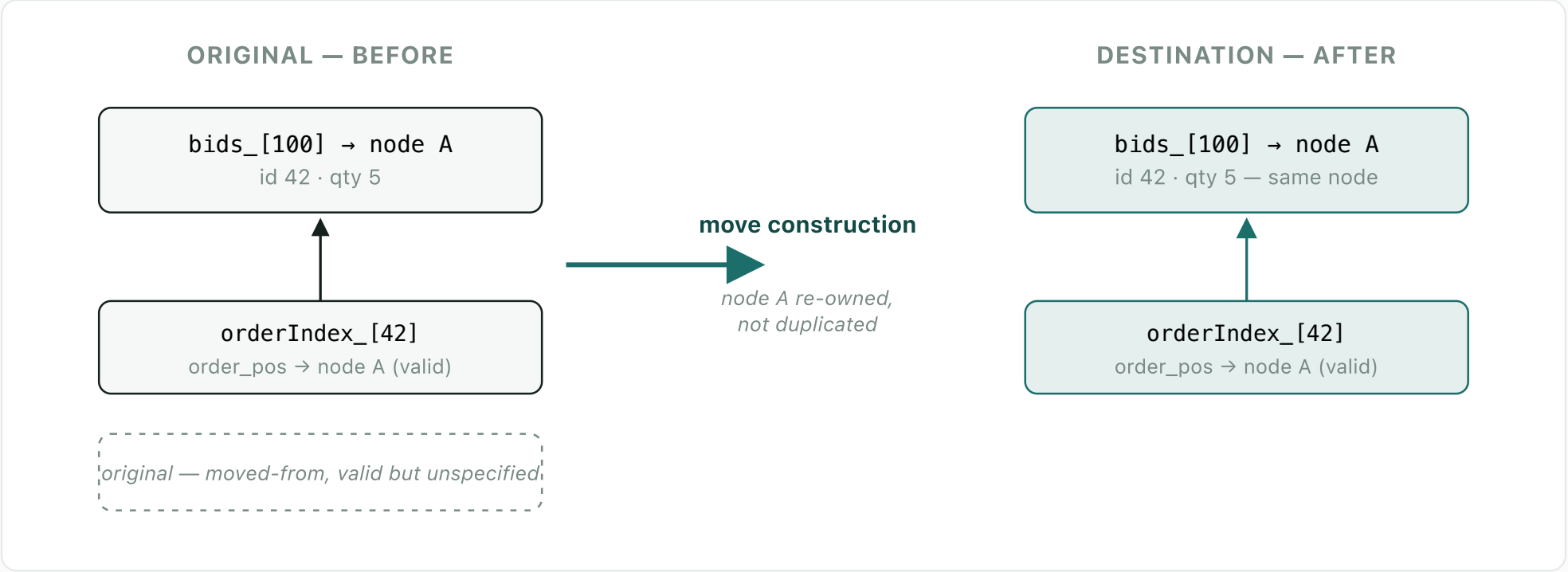

Figure 3. Contrast with Figure 2: no edge crosses to the wrong side. Node A physically relocates once; both of `destination`'s views agree on it, because they were never duplicated relative to each other in the first place.

```
OrderBook(OrderBook&&)            = default;   // move construction
OrderBook& operator=(OrderBook&&) = default;   // move assignment
```

A PRECISE POINT

`std::move` does not physically move anything by itself. It is a cast — it tells the compiler *you may treat this object as a source to transfer from*, which makes the move constructor eligible for overload resolution. The actual node transfer happens inside `std::map` / `std::list`'s own move constructors, which `= default` simply delegates to member-by-member.

After a move: `destination` owns the book's state; `original` remains destructible and assignable, but nothing should be assumed about which orders, if any, still appear inside it. This will matter directly once snapshot seeding constructs a complete book and hands it to a replay controller — the natural way to make that handoff is a move, not a copy.

5 Why the default constructor had to be spelled out

```
OrderBook() = default;
```

On its own this line changes nothing — an empty book built from value-initialised maps is what would happen anyway. What it buys is legibility: once a class starts declaring *any* special member function, a reader can no longer assume the rest are still the ordinary compiler-generated ones. Writing `= default` here states the ownership policy in full, in one place, rather than leaving one part of it implicit.

6 Why the move operations had to be written, not assumed

This is the one genuinely sharp corner in the whole design. **Declaring a copy constructor — even a deleted one — stops the compiler from generating move operations for you.** Had

the header stopped at:

```
OrderBook(const OrderBook&) = delete;
```

… `OrderBook` would silently end up *neither* copyable *nor* movable — a strictly worse outcome than doing nothing at all, since Slice 2's own plan to move a completed book into a replay controller would stop compiling with no obvious reason why. Spelling out every operation closes that gap. This is the classic **Rule of Five**: once you touch one of these five, state your intent for all five.

OPERATION	DECLARED AS	WHY
`~OrderBook()`	*not declared*	Standard containers clean up their own nodes; nothing custom to release.
`OrderBook(const OrderBook&)`	**= delete**	A default copy would leave locators pointing into the wrong book (Figure 2).
`operator=(const OrderBook&)`	**= delete**	Same hazard, on assignment into an already-live book.
`OrderBook(OrderBook&&)`	**= default**	Standard containers transfer nodes on move; every locator stays correct (Figure 3).
`operator=(OrderBook&&)`	**= default**	Same guarantee, on assignment.

7 What the `static_assert`s do

```
static_assert(!std::is_copy_constructible_v<OrderBook>, "...");
static_assert(!std::is_copy_assignable_v<OrderBook>,    "...");
static_assert( std::is_move_constructible_v<OrderBook>, "...");
static_assert( std::is_move_assignable_v<OrderBook>,    "...");
```

Each of these is a compile-time question put directly to the compiler — *"can an* `OrderBook` *be copy-constructed?"* — with the answer asserted, not assumed. They belong beside the

class itself, because this is a permanent property of the *type*, not a fact about any particular build or call site. If someone later deletes the deleted-copy lines thinking they're dead code, the assertion fails at compile time and the message explains exactly why, right where the mistake was made — rather than as a dangling-iterator bug reported from wherever Modify or Remove eventually happened to run.

> **VERIFIED** `clang++ -std=c++20 -Wall -Wextra -fsyntax-only order_book.hpp` — compiles clean. The four assertions above are not aspirational prose; they hold against the header as it stands in `order_book.hpp:68–79` right now.

8 Where else this pattern already lives

The question to ask when designing any class: *if I duplicated every member, would the new object be completely independent and internally correct?* For plain values the answer is trivially yes. For anything that owns a resource, or encodes relationships between its own members the way `OrderBook` does, it usually isn't — and this codebase already has two other answers to the same question.

TYPE	WHAT IT OWNS	COPY	MOVE	WHY
`OrderBook`	Interconnected containers — locators hold iterators into its own lists.	**deleted**	**defaulted**	A copy's iterators would still resolve into the original book (Figure 2).
`Sink` `telemetry/sink.hpp:115`	An open output file stream.	**deleted**	**defaulted**	Two `Sink`s must not both believe they own the same stream — who flushes it, who closes it?
`TempFile` `test_recorder.cpp:24`	A path on disk, removed by its own destructor.	**deleted**	*not declared*	Two copies would each try to delete the same file; the second delete is a bug waiting to happen.

The same one-line reasoning explains all three: **one resource, one clear owner.** For `Sink` the resource is a file stream. For `TempFile` it's a path on disk. For `OrderBook` it's a set of containers wired together by iterators — a resource made of relationships rather than a handle, but a resource all the same. `TempFile` stops one step short of the other two only

because nothing in its current use ever needs to hand ownership onward; there was no move to write.

THE GENERAL LESSON

"If I duplicate every member, will the new object be completely independent and internally correct?"

Copying is appropriate for independent values that carry no relationship to anything else — `Price`, `Qty`, `OrderId`, `OrderEvent`. Duplicate every field and you get two objects that owe each other nothing.

Moving, or unique ownership, is appropriate for anything that owns a resource or encodes relationships between its own members — `Sink`, `TempFile`, `OrderBook`. For `OrderBook` specifically, the answer to the question above is no, because its iterators exist precisely to encode a relationship between `orderIndex_` and the lists inside `bids_` / `asks_`. Copying is forbidden — permanently, at compile time — unless a future change deliberately writes a custom deep copy that rebuilds every locator against the new lists from scratch.